

TA-IND-04 — Análisis de Rendimiento Paralelo aplicado al PFC

Transformación de foco individual: T1 (Filtrado y selección)

Robinson Rodrigo Cando Moreno

8 de agosto de 2026

Universidad: Universidad Técnica Estatal de Quevedo

Facultad: Ciencias de la Computación — **Carrera:** Ingeniería de Software

Asignatura: Aplicaciones Distribuidas (ISR-701) — **Unidad 4**

Equipo de PE-U4: Cristhian Daniel Pacheco Cárdenas, Robinson Rodrigo Cando Moreno, Ernesto Gregory Luna Mora

PFC de referencia: ACC — Sistema de Gestión de Soporte Técnico ISP

Transformación declarada como foco individual: T1 — Filtrado y selección

1. Introducción y contexto

Este informe analiza de forma individual los resultados experimentales de la práctica GA-SUM-05/PE-U4, en la que el equipo comparó cinco transformaciones (T1–T5) sobre un dataset sintético de 600 000 tickets de soporte técnico ISP, implementadas en `pandas` y en `PySpark 3.5.3`, con un protocolo de un calentamiento más cinco repeticiones cronometradas (mediana como estadístico). El objetivo de este documento no es repetir esas mediciones, sino someterlas a un análisis cuantitativo propio frente al modelo de Amdahl [1] — y, donde corresponda, frente a su reformulación en términos de carga escalada de Gustafson [2] — y a la métrica de Karp-Flatt [3], y decidir, con ese diagnóstico, cómo integrar procesamiento distribuido en el PFC de referencia (ACC — Sistema de Gestión de Soporte Técnico ISP).

Origen de los datos. Los datos base del equipo provienen del repositorio <https://github.com/CristhianP03/pe-u4-spark-equipoA>, commit 19e84a0072f1fe49a53d1e5b71b17ba8d83091b3 (archivo `resultados/tiempos_resumen.csv`). Ese commit escala únicamente T3 en $N = 1, 2, 4$; para el resto de transformaciones solo mide $N = 4$. Como mi foco individual es **T1**, que no estaba escalado, reproduce el pipeline completo del equipo (mismo script `medicion.py`, mismo protocolo) en una sesión propia de Google Colab el 8 de agosto de 2026, y además ejecuté mediciones adicionales de T1 en $N = 1$ y $N = 2$ (que el script del equipo no contempla por defecto). Todos los valores de este informe — T1 a T5 — provienen por lo tanto de esa única sesión de reproducción, internamente consistente entre sí, y se incluyen en `datos/tiempos_base.csv` de este repositorio. Esta decisión se declara aquí de forma explícita para no mezclar cifras de sesiones de medición distintas.

2. Análisis propio de los resultados de *speedup*

2.1. Tabla de datos base

La tabla 1 resume las medianas de cinco repeticiones para las cinco transformaciones (`pandas` y `Spark` con $N = 4$), y el escalado propio de T1 en $N = 1, 2, 4$.

Aquí $S_{\text{interno}}(N) = t_{\text{spark}}(N=1)/t_{\text{spark}}(N)$ es el escalado *interno* del motor (Ec. 1), el que corresponde comparar con la curva de Amdahl, mientras que $S_{\text{vs. pandas}}$ es la aceleración global frente a la línea base secuencial.

Cuadro 1: Tiempos medianos (5 repeticiones) y *speedup* vs. pandas, $N = 4$.

Transformación	T_{pandas} (s)	T_{spark} (s)	S	E ($N = 4$)
T1 — Filtrado y selección (foco)	8,352	5,777	1,446	0,361
T2 — Agrupación y agregación	6,705	3,576	1,875	0,469
T3 — <i>Join</i> de dos DataFrames	8,895	7,055	1,261	0,315
T4 — Columna derivada compleja	12,264	7,069	1,735	0,434
T5 — Ordenamiento y <i>top-N</i>	6,813	2,372	2,872	0,718

Cuadro 2: Escalado propio de T1 (foco) en Spark, $N = 1, 2, 4$. Base pandas: 8,352 s.

N	$T_{\text{spark}}(N)$ (s)	$S_{\text{vs. pandas}}(N)$	$S_{\text{interno}}(N)$	$E(N)$
1	5,285	1,580	1,000	1,000
2	5,012	1,666	1,054	0,527
4	5,777	1,446	0,915	0,229

2.2. Ley de Amdahl y eficiencia

Con p la fracción paralelizable del programa y N el número de unidades de procesamiento, la Ley de Amdahl [1] predice:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p}. \quad (1)$$

La eficiencia del paralelismo se define como

$$E(N) = \frac{S(N)}{N}. \quad (2)$$

2.3. Contraste con la predicción teórica

La tabla 2 muestra un patrón **no monótono**: el *speedup* interno de T1 *mejora* de $N = 1$ a $N = 2$ (S_{interno} de 1.000 a 1.054), pero *empeora* en $N = 4$, cayendo a 0,915 — es decir, con 4 executors T1 corre más lento que con uno solo. La figura 1 lo muestra gráficamente.

¿Coinciden los resultados con la predicción de Amdahl? No, y de una forma más severa que una simple desviación por defecto. La Ley de Amdahl asume sobrecarga nula: bajo esa hipótesis, añadir procesadores *nunca* puede hacer que un programa corra más lento, de modo que el límite inferior teórico es $S(N) \geq 1$ para toda $N \geq 1$ (el caso $p = 0$, totalmente serial). El valor observado $S_{\text{interno}}(4) = 0,915$ está *por debajo de ese piso teórico*, lo cual el modelo de Amdahl no puede representar bajo ninguna combinación de $p \in [0, 1]$ — es matemáticamente imposible en el modelo ideal. Tomando como referencia más cercana el caso límite $p = 0$ ($S_{\text{teo}} = 1$), la desviación relativa es

$$\Delta = \frac{S_{\text{med}} - S_{\text{teo}}}{S_{\text{teo}}} = \frac{0,915 - 1}{1} = -8,5 \%. \quad (3)$$

Como explica la literatura sobre los límites prácticos de la ley en arquitecturas reales [4], esto ocurre precisamente porque Amdahl no modela la sobrecarga de coordinación: cuando el trabajo por transformar es pequeño (T1 es un filtro de una sola pasada, sin *shuffle*), el costo fijo de orquestar más executors (planificación de tareas, más particiones, comunicación *driver-executor*) puede superar cualquier beneficio de paralelizar. La sección 3 cuantifica esta sobrecarga con Karp-Flatt y la atribuye a mecanismos concretos.

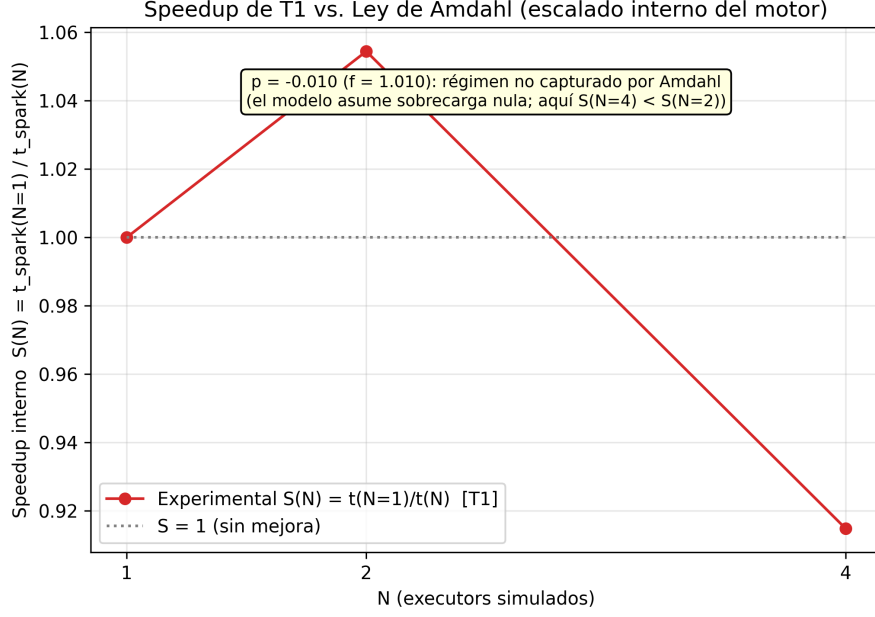


Figura 1: *Speedup* interno experimental de T1 frente a la curva de Amdahl. El ajuste de Karp-Flatt entre $N = 2$ y $N = 4$ produce $p < 0$, fuera del dominio válido del modelo, por lo que no se traza una curva teórica.

3. Diagnóstico de la fracción no escalable

La métrica de Karp-Flatt [3] estima experimentalmente la fracción serial e a partir del *speedup* observado con N unidades:

$$e = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1. \quad (4)$$

Sustituyendo los valores de la tabla 2:

$$e(N=2) = \frac{\frac{1}{1,054} - \frac{1}{2}}{1 - \frac{1}{2}} = \frac{0,9483 - 0,5}{0,5} = 0,897,$$

$$e(N=4) = \frac{\frac{1}{0,915} - \frac{1}{4}}{1 - \frac{1}{4}} = \frac{1,0931 - 0,25}{0,75} = 1,124.$$

Tendencia: e crece de 0,897 ($N=2$) a 1,124 ($N=4$) — de hecho supera 1, un valor fuera del rango teórico $[0, 1]$ de una fracción serial "pura". Según el criterio de interpretación de Karp y Flatt [3], un e que crece con N indica que la pérdida de escalabilidad **no** es un límite algorítmico fijo de T1 (en ese caso e se mantendría aproximadamente constante), sino **sobrecarga creciente del motor de ejecución**. Que e supere 1 refuerza esta lectura: no es que T1 tenga una fracción serial intrínsecamente alta, es que el costo de coordinar más executors crece más rápido que el trabajo que se logra paralelizar.

Atribuyo esta sobrecarga a tres mecanismos concretos de Spark, consistentes con la descripción arquitectónica del motor [5]:

1. **Planificación del DAG y lanzamiento de tareas.** Spark hereda del paradigma de procesamiento por lotes popularizado por MapReduce [6] el principio de dividir el trabajo en tareas independientes por partición; cada executor adicional implica más tareas que el *driver* debe planificar, serializar y enviar. Para una transformación de una sola etapa como

T1 (filtro simple, sin *shuffle*), este costo fijo por tarea pesa proporcionalmente más que en transformaciones con más cómputo por partición.

2. **Arranque y coordinación de executors.** Cada sesión Spark nueva (`crear_spark(n)`) reinicia executors JVM independientes; con $N=4$ hay más JVMs que sincronizar (*heartbeats*, gestión de memoria) que con $N=1$, y ese costo es esencialmente fijo, no decrece con el volumen de datos por executor.
3. **Entorno compartido y con ruido (Google Colab).** A diferencia de un clúster dedicado, Colab ejecuta sobre infraestructura multiinquilino con variabilidad de CPU/E-S entre sesiones; a mayor número de procesos concurrentes (más executors), mayor exposición a esa variabilidad, lo que puede amplificar el efecto de los dos mecanismos anteriores en la mediana observada.

4. Propuesta de integración al PFC

Caso de uso. En el PFC ACC (Sistema de Gestión de Soporte Técnico ISP), el punto de inserción propuesto es un **proceso por lotes nocturno** que recorre todos los tickets creados/actualizados en el día y calcula, por zona y por categoría: cumplimiento de SLA, tiempo medio de respuesta y desempeño por agente — operaciones análogas a T2 (agrupación) y T4 (columna derivada), que en este mismo experimento mostraron los mejores *speedups* (1.875 y 1.735 respectivamente, tabla 1), a diferencia de T1.

Arquitectura de integración. El sistema ACC sigue una arquitectura en capas (presentación web, API de negocio, base de datos relacional). El job de Spark se inserta *fuera* del camino síncrono de la API, como un proceso batch independiente:

1. Un disparador programado (*cron*, 02:00 AM) lanza el job.
2. El job lee los tickets del día directamente de la base de datos (extracción de solo lectura, réplica o ventana de mantenimiento).
3. Ejecuta las agregaciones por zona/categoría/agente (equivalentes a T2/T4) y escribe los resultados en una tabla de reportes agregados (`reportes_diarios`), no en las tablas transaccionales.
4. El panel de supervisión (capa de presentación) consulta esa tabla agregada, no el job de Spark directamente.

Decisión que soporta. La salida alimenta un panel de supervisores que decide, semana a semana, a qué zona o equipo reforzar con más agentes según el cumplimiento de SLA observado — una decisión operativa concreta, no un reporte genérico.

Por qué no T1 en este punto. Es deliberado que el proceso batch propuesto se parezca a T2/T4 y no a T1: el hallazgo de la sección 3 indica que filtros ligeros de una sola pasada como T1 no se benefician de paralelizar con Spark en volúmenes de esta magnitud; ese tipo de operación es más apropiado ejecutarla de forma síncrona con *pandas* (o directamente como consulta SQL) dentro de la propia API, no como job distribuido.

5. Dimensionamiento y viabilidad

Modelando el tiempo secuencial como $T_{\text{sec}}(n) = \alpha n$ y el distribuido como $T_{\text{dist}}(n) = \beta + \gamma n/N$, el umbral de rentabilidad es

$$n^* = \frac{\beta}{\alpha - \frac{\gamma}{N}}, \quad \text{válido si } \alpha > \frac{\gamma}{N}. \quad (5)$$

Método de ajuste (con su limitación declarada). No se dispone, para T1, de mediciones a distintos tamaños de dataset (el equipo solo escaló volumen para T2, en el modo `umbral` de `medicion.py`); para T1 solo se midió el dataset completo (600 000 filas) en $N = 1, 2, 4$. Ante esa limitación, estimo α directamente de la línea base pandas ($\alpha = 8,352/600\,000 = 1,392 \times 10^{-5}$ s por registro) y ajusto β, γ por regresión lineal de T_{dist} sobre $1/N$ usando únicamente $N = 1$ y $N = 2$ — excluyo deliberadamente $N = 4$ del ajuste porque en ese punto domina el efecto de sobrecarga no lineal diagnosticado en la sección 3, que el modelo lineal no captura:

$$\gamma = \frac{5,285 - 5,012}{600\,000 - 300\,000} = 9,10 \times 10^{-7} \text{ s}, \quad \beta = 5,285 - 600\,000 \gamma = 4,739 \text{ s}.$$

Con $N = 4$: $\gamma/N = 2,28 \times 10^{-7}$, que cumple $\alpha > \gamma/N$, y

$$n^* = \frac{4,739}{1,392 \times 10^{-5} - 2,28 \times 10^{-7}} \approx 346\,000 \text{ registros}.$$

Lectura crítica del propio resultado. Este n^* (346 000) es *menor* que el tamaño real del dataset (600 000), es decir, el modelo lineal — ajustado solo con el tramo "bien comportado" ($N=1 \rightarrow 2$) — predeciría que Spark ya debería ser rentable a escala completa. Sin embargo, la medición real en $N = 4$ contradice esa predicción ($S_{\text{interno}} < 1$). Esta contradicción es en sí misma un resultado relevante: confirma que, para transformaciones ligeras como T1, un modelo de costo lineal en n es insuficiente para decidir el umbral de rentabilidad; se necesita, además, monitoreo empírico del comportamiento del motor a distintos N , no solo a distintos n .

Modelo de servicio en la nube. Dado que ACC es un sistema ISP con carga diaria predecible (job nocturno, no interactivo), recomiendo un clúster **efímero bajo demanda** (p. ej. Databricks Community/Job Clusters o EMR en modo *transient*) que se aprovisiona solo para la ventana del job y se apaga al terminar, en vez de un clúster permanente: minimiza costo fijo cuando, como muestra este análisis, el beneficio marginal de más executors no está garantizado para todas las transformaciones del pipeline.

Riesgos. (i) Variabilidad de rendimiento en infraestructura compartida, igual que la observada en Colab; (ii) sobre-aprovisionar executors para operaciones ligeras, replicando el problema de T1 en producción; (iii) ventana de mantenimiento nocturna insuficiente si el volumen de tickets crece más rápido de lo proyectado.

6. Postura crítica y alternativas

¿Es Spark la opción correcta para el PFC? Con matices: sí para el proceso batch propuesto en la sección 4 (agregaciones tipo T2/T4, con *speedups* de 1.7–1.9× observados), pero **no** como motor universal para todo el pipeline de datos del sistema — el propio T1 demuestra que, para operaciones ligeras, la sobrecarga de Spark puede ser contraproducente.

Alternativa 1: Dask. Biblioteca de cómputo paralelo en Python puro. Se descarta como reemplazo general porque, aunque evita la sobrecarga JVM identificada en la sección 3 (mecanismo 2) y sería razonable para T1, su ecosistema de conectores y su madurez en *joins* distribuidos a gran escala es menor que el de Spark [5]; un ISP en crecimiento necesita esa capacidad para las operaciones pesadas (T2/T3/T4), donde Spark ya demuestra ventaja.

Alternativa 2: escalar verticalmente pandas (sin motor distribuido). Los propios datos de este informe muestran que pandas procesa las 600 000 filas en 6–12 segundos según la transformación — perfectamente aceptable para un job nocturno hoy. Se descarta como estrategia de *largo plazo* porque no escala horizontalmente: el volumen de tickets de un ISP crece con su base de clientes, y verticalizar hardware indefinidamente no tiene el mismo techo de costo/beneficio que un clúster elástico. Es, sin embargo, la opción correcta *por debajo* del umbral n^* estimado en la sección anterior — lo cual es coherente con el propio resultado cuantitativo de este informe, no una preferencia arbitraria.

7. Conclusiones

1. Para T1, la sobrecarga de Spark supera su beneficio de paralelismo en $N = 4$ dentro del rango probado ($S_{\text{interno}}(4) = 0,915 < 1$), invalidando la suposición ingenua de que más executors siempre aceleran una transformación (evidencia: tabla 2, figura 1).
2. La tendencia creciente de Karp-Flatt (e de 0.897 a 1.124 entre $N = 2$ y $N = 4$) muestra que la no escalabilidad observada se debe a sobrecarga *creciente* del motor, no a un límite algorítmico fijo de T1 (evidencia: Ecuación 4 aplicada en la sección 3).
3. El procesamiento distribuido en el PFC debe reservarse para operaciones agregadas y pesadas (tipo T2/T4, *speedups* 1.7–1.9 $\times$), no para filtros ligeros por registro como T1; esto determinó directamente el punto de inserción arquitectónico propuesto en la sección 4 (evidencia: tabla 1, comparación de *speedups* entre las cinco transformaciones).

8. Declaración de uso de inteligencia artificial generativa

Se utilizó Claude (Anthropic) como asistente durante la elaboración de este informe, con los siguientes propósitos: (i) orientación paso a paso para reproducir el pipeline de PE-U4 en Google Colab y generar las mediciones propias de T1 en $N = 1, 2$; (ii) generación del código Python (matplotlib) para la figura propia de la sección 3 a partir de los tiempos medidos por el autor; (iii) redacción asistida del borrador de las secciones 1 a 8, sobre la base de los datos, cálculos y decisiones de diseño aportados por el autor. Los cálculos numéricos, la interpretación de los resultados y las decisiones de arquitectura fueron revisados por el autor antes de la entrega.

Referencias

- [1] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,” en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67 (Spring))*, Atlantic City, NJ, USA: ACM, 1967, págs. 483–485. DOI: 10.1145/1465482.1465560
- [2] J. L. Gustafson, “Reevaluating Amdahl’s Law,” *Communications of the ACM*, vol. 31, n.º 5, págs. 532–533, 1988. DOI: 10.1145/42411.42415
- [3] A. H. Karp y H. P. Flatt, “Measuring Parallel Processor Performance,” *Communications of the ACM*, vol. 33, n.º 5, págs. 539–543, 1990. DOI: 10.1145/78607.78614
- [4] M. D. Hill y M. R. Marty, “Amdahl’s Law in the Multicore Era,” *Computer*, vol. 41, n.º 7, págs. 33–38, 2008. DOI: 10.1109/MC.2008.209
- [5] M. Zaharia et al., “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, n.º 11, págs. 56–65, 2016. DOI: 10.1145/2934664
- [6] J. Dean y S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” *Communications of the ACM*, vol. 51, n.º 1, págs. 107–113, 2008. DOI: 10.1145/1327452.1327492